

Lesson 17: Animations

Why small animations matter

Open two apps: one where screens jump instantly, and one where things glide, fade, and grow. The second one *feels* better -- even if both do the same job. Small animations tell the user what just happened: "this card opened", "that item was added", "you moved to a new screen".

The good news: in Flutter, the easiest animations need almost no extra code.

Implicit animations: describe the end, Flutter moves

Implicit animations are the simplest kind. You say what the widget should *look like*, and when the values change, Flutter moves smoothly from old to new on its own.

The most useful one is `AnimatedContainer`. It works exactly like `Container`, but changes to its size, color, or shape animate over time:

```
AnimatedContainer(  
  duration: const Duration(milliseconds: 500),  
  width: isBig ? 200 : 100,  
  height: isBig ? 200 : 100,  
  color: isBig ? Colors.blue : Colors.red,  
  child: const Text('Tap me'),  
);
```

When `isBig` flips (say, on a button tap with `setState`), the box grows *and* changes color smoothly over half a second. You wrote no animation code -- just the start and end values.

Other widgets in the same family: `AnimatedOpacity` (fade in/out), `AnimatedPadding`, `AnimatedAlign`. Same idea every time: change the value, Flutter animates the move.

Hero animations: one widget, two screens

A Hero animation is the smooth "flying" effect you see when you tap a small image and it grows into the full details screen. Both screens share one widget that *flies* between them.

You only need two things:

1. Wrap the widget on both screens in a `Hero` with the same tag.
2. That is it. Flutter does the flight.

```
// On the list screen:  
Hero(  
  tag: 'product-image',
```

```
child: Image.network('https://example.com/shoe.png'),  
);  
  
// On the details screen:  
Hero(  
  tag: 'product-image',  
  child: Image.network('https://example.com/shoe.png'),  
);
```

When you push the details screen, the image lifts off the list and lands in its new spot. The matching tag is what links the two -- use one tag per flying widget.

A sense of timing

Two small rules make animations feel right:

- Keep them short. 200-500 milliseconds is the sweet spot. Longer feels slow; shorter feels like a jump.
- Animate with meaning. Animate the thing that changed -- the added item, the opened card. Random motion is just noise.

You do not need complex animation controllers yet. `AnimatedContainer` and `Hero` cover most of what a beginner app needs.

Key terms

- Implicit animation -- you describe the end look; Flutter moves smoothly from old to new on its own.
- `AnimatedContainer` -- a container that animates changes to size, color, and shape.
- `Hero` -- a widget that flies from one screen to another during navigation, linked by a matching tag.
- Duration -- how long an animation takes, often 200-500 milliseconds.